Stela System Architecture Refinement: Operational Hardening, Constitutional Alignment, and the Pointer-First Doctrine

1. Executive Summary

This report presents a definitive architectural refinement strategy for the Stela repository-resident artificial intelligence system. It addresses critical maturity gaps identified in the system's operational lifecycle, specifically concerning the "Closeout" phase of Dispatch Packets (DPs), the governance of intellectual capital through automated harvesting, and the resolution of jurisdictional conflicts regarding Single Source of Truth (SSoT) branch naming conventions. The analysis is predicated on a deep forensic review of the system's current state—specifically HEAD d14969db—and synthesizes these findings into a rigorous "Hardening Plan" that elevates the system from a managed workflow to a self-correcting, immunologically robust organism.

The central thesis of this report is that a repository-resident AI cannot rely on narrative documentation alone; it must operate on a **"Pointer-First" Constitution**. In this architecture, the active work surface (TASK.md) and the governance layer (PoT.md) do not contain ephemeral instructions but rather immutable pointers to executable logic (ops/) and canonical definitions (docs/). This minimizes "Canon Drift"—the inevitable divergence between written rules and executed reality—which is identified as a primary failure state in the system's Policy of Truth.[1]

Key deliverables detailed in this report include:

1. **Resolution of SSoT Conflict**: A constitutional amendment to PoT.md to canonize branch naming conventions, resolving the "Jurisdictional Drift" currently observed between the constitutional core and ephemeral instruction sets.[1]
2. **Hardened Closeout Cycle**: The definition of a mandatory, automated four-stage Closeout Cycle (Verify, Harvest, Refresh, Prune) that integrates ops/bin/llms for truth synchronization and ops/lib/scripts for knowledge capture.[1]
3. **Operational Binary Refinement**: Detailed specifications for modifying ops/bin/prune to support "DP Cleaning"—a targeted hygiene protocol to remove metabolic waste (intermediate artifacts) while preserving the repository's structural integrity.[1]
4. **Pattern Density Harvesting**: An analysis of the "Harvest Engine" logic, utilizing pattern density heuristics to convert implicit agent behavior into explicit, reusable Skills and Tasks.[1]

This document is structured to serve as the authoritative implementation guide for the "Stela Hardening" initiative. It integrates theoretical underpinnings of AI governance with precise, line-level operational specifications, ensuring that the system's evolution aligns with its foundational axioms of "Linguistic Precision," "Reuse-First," and "Absolute Literalism".[1]

---

2. Theoretical Framework: The Repository as a Constitutional State

To understand the necessity of the proposed hardening measures, one must first appreciate the unique theoretical framework of the Stela system. Unlike traditional software repositories where governance is often extrinsic (human managers, external wikis), Stela operates as a **Repository-Resident AI**. In this model, the repository is not merely a storage container for code; it is the "Environment" in which the agent lives, acts, and is governed.

2.1 The Policy of Truth (PoT) as Physical Law

The PoT.md file is defined not as a guideline, but as the "single constitutional document" and "final authority".[1] This distinction is critical. In AI governance, a "guideline" is a suggestion that can be outweighed by probabilistic weighting in a Large Language Model (LLM). A "Physical Law," however, is a constraint that triggers a system halt if violated.

The current forensic analysis reveals that Stela employs a **"Doctrine of Prevention"** rather than "Compliance".[1] The system is designed to stop work, reject PRs, and kill processes when drift is detected. This alignment with broader AI safety research—which emphasizes structural transparency and institutional constraints over mere informational transparency—is evident in the rigorous enforcement mechanisms detailed in the repository's workflows.[2] However, a constitution is only effective if it is complete. The identified gaps in branch naming governance [1] represent a "Constitutional Void" where the agent is liable to hallucinate or invent rules, leading to the failure state of "Ambiguity" defined in PoT Section 2.[1]

2.2 The "Pointer-First" Philosophy

The "Pointer-First" architecture is the system's defense against "Hallucination" and "Drift." In early-stage AI systems, instructions are often duplicated: a rule about "no contractions" might appear in the prompt, the readme, and the linter config. Over time, these diverge.

- **Narrative Architecture**: "Always run npm test before committing." (Fragile: What if the command changes to npm run verify?)
- **Pointer-First Architecture**: "Execute the Verification Pointer defined in TASK.md." (Robust: TASK.md points to tools/verify.sh. If the script changes, the pointer remains valid).

By refining TASK.md to be a dashboard of pointers rather than a container of text [1], we ensure that the agent always executes the *current* version of the law. This mirrors the "Three-Pillar Approach" to AI accountability found in external governance frameworks, which emphasizes identifying responsible entities (Agents), articulating expectations (Pointers), and auditing actions (Verification).[3]

---

3. Governance Hardening: Resolving the SSoT Conflict

A critical finding during the forensic review of the dump-full-main-d14969db artifact is a violation of the Single Source of Truth (SSoT) axiom regarding branch naming conventions.[1]

3.1 Forensic Analysis of Jurisdictional Drift

The Policy of Truth (PoT.md) explicitly claims to be the "sole authority across all scopes" and states that "No parallel jurisdictions exist".[1] However, the rules governing where work can be performed (i.e., branch naming) are absent from PoT.md.

1. **The Constitutional Void**: PoT.md Section 1.1 defines projects/ as the directory for "Work" but remains silent on the Git branch nomenclature.[1] Section 2 mentions main only to define the "Dirty State" exemption for TASK.md [1], implying main is a monitored state but not explicitly creating a "Physical Law" against pushing to it within the text of the constitution itself.
2. **The Hidden Statute**: The operational prohibition is sequestered in .github/copilot-instructions.md, a lower-tier document not listed in the mandatory "Context Load" of TASK.md.[1] This file contains the "Hard Rule": *"No Direct Pushes: Do not attempt to push to main. All work occurs on work/* branches"*.[1]

This separation creates **Jurisdictional Drift**. The instruction file is effectively legislating a rule that the constitution ignores. If an agent prioritizes PoT.md (as commanded by the Read-in Order), it might technically infer that working on main is permissible provided it avoids "Dirty State" triggers, directly violating the hard rule in the instructions. This ambiguity is a "System Failure State".[1]

3.2 Strategic Resolution: The Branching Doctrine Amendment

To resolve this conflict and harden the SSoT, the branching rules must be elevated from "Instruction" to "Law." We propose a constitutional amendment to PoT.md to explicitly define the "Branching Doctrine."

3.2.1 Proposed Text for PoT.md Amendment

The following text should be integrated into PoT.md, likely under **Section 5: Workflow &**

Security.

5.5 Branching Doctrine (SSoT)

- **Immutable Trunk**: The main branch is reserved for verified, deployable state. Direct pushes to main are constitutionally forbidden.
- **Work Namespace**: All active development must occur on branches prefixed with work/.
- **Naming Schema**: Valid formats are work/<dp-id>-<topic> or work/<topic>-YYYY-MM-DD.
- **Drift Prevention**: Any branch not matching this schema is considered "Trash" (per Filing Doctrine) and subject to immediate pruning.

By embedding this rule in PoT.md, we satisfy the SSoT axiom. The .github/copilot-instructions.md file must then be refactored to simply *point* to PoT.md Section 5.5, ensuring that the instruction file remains a "pointer" rather than a container of unique truth. This aligns with the "Reuse-first" axiom.[1]

---

4. The Refined Closeout Cycle: Operational Hardening

The core of the hardening plan is the re-engineering of the **Closeout Cycle**. In the current state, TASK.md acts as a manual checklist. The goal is to transform Closeout into an automated, "gate-kept" pipeline that integrates **Harvesting**, **Refreshing**, and **Pruning**.

4.1 The Need for a Metabolic Cycle

An AI system "metabolizes" tasks. It ingests context (OPEN), processes it (work/), and produces output (PR). However, without a formal metabolic cycle, two toxic byproducts accumulate:

1. **Metabolic Waste**: Intermediate files (OPEN-PORCELAIN, dump-full) clutter the storage/ directory, consuming disk space and confusing future context retrieval.[1]
2. **Uncaptured Knowledge**: New patterns or workflows discovered during execution remain implicit in the Git history rather than being explicit in the Library (docs/library/).

The Refined Closeout Cycle addresses these by enforcing a four-stage process: **Verify -> Harvest -> Refresh -> Prune**.

4.2 Stage 1: Verification (The Gate)

Before any closeout actions, the work must be proven valid using the system's "Immune System" tools.[1]

- **Mechanism**: The TASK.md Closeout section must point to the following mandatory commands:

    - bash tools/verify.sh: Ensures filing doctrine compliance (no binaries in docs/).[1]
    - bash tools/lint/truth.sh: Validates "Truth Integrity" and checks for typos of "Stela" or prohibited contractions.[1]
    - bash tools/lint/library.sh: Enforces the integrity of the docs/library/ and docs/ops/registry/.[1]

4.3 Stage 2: Harvest (Memory Consolidation)

This is the critical "Learning" phase. The system must analyze its own recent behavior to extract reusable assets.

- **Pattern Density Analysis**: The ops/lib/scripts/agent.sh script includes a harvest-check mode.[1] This heuristic analyzes the "Pattern Density"—clusters of tools and pointers used repeatedly in recent DPs.[1]
- **The Mandate**: If harvest-check identifies a cluster with high recurrence (e.g., used in >3 recent DPs), the Integrator is *obligated* to harvest it as a new Agent or Skill.
- **Execution**:
    - **Skill Harvest**: ops/lib/scripts/skill.sh harvest --name "...".[1] This creates a draft skill in storage/archives/skills/.
    - **Task Harvest**: ops/lib/scripts/task.sh harvest --id B-TASK-XX....[1]
- **Registry Enforcement**: The harvest scripts automatically update the SSoT registries (docs/ops/registry/SKILLS.md), ensuring that the new knowledge is immediately discoverable.[1]

4.4 Stage 3: Refresh (Truth Synchronization)

A major risk in AI development is the "Stale Truth" hazard—where the documentation (Map) lags behind the code (Territory).

- **The Tool**: ops/bin/llms. This binary is the "Context Bundler".[1] It compiles the canonical surfaces into optimized bundles (llms-small.txt, llms-full.txt, llms-governance.txt).[1]
- **The Protocol**: The Closeout cycle must mandate running ops/bin/llms *immediately* after any modification to a file listed in ops/lib/manifests/CONTEXT.md.[1]
- **Flat Truth Enforcement**: The binary writes these bundles to the repository root. This "Flat Truth" visibility is crucial for preventing hallucinations, as it forces the context window to accept the *generated* reality over any *inferred* reality.[1]

4.5 Stage 4: Prune (Hygiene & Metabolic Clearance)

The final stage is the removal of waste. The ops/bin/prune binary must be modified to support targeted "DP Cleaning".[1]

- **Current State**: The storage/ directory is defined as "Trash" [1], but unmanaged trash leads to "Storage Drift" (unexpected clutter).[1]
- **The Modification**: ops/bin/prune must accept a --dp=<DP-ID> flag.

- **Logic**: It scans storage/handoff/ and storage/dumps/ for files matching the DP ID (e.g., OPEN-*-DP-OPS-XXXX.txt, dump-*-DP-OPS-XXXX.txt).
- **Safety**: It must strictly preserve .gitkeep files to maintain directory structure.[1]
- **Reporting**: It must output "Pruned X files..." (avoiding contractions like "It's done" to satisfy Linguistic Precision [1]).

---

5. Technical Specifications: Operational Binaries

To implement the Hardened Closeout Cycle, specific modifications are required to the operational binaries. This section provides the technical specifications for these changes, adhering to the "Behavioral Logic Standard" of the PoT.

5.1 Specification: ops/bin/prune (Modification)

Purpose: To surgically remove intermediate artifacts associated with a specific Dispatch Packet without disrupting the repository's structural integrity.

New Requirement: DP-Scoped Cleaning

- **Input**: The binary shall accept a flag --dp=<DP-ID> (e.g., --dp=DP-OPS-0046).
- **Target Selection**:
  - Scan storage/handoff/ for files matching *<DP-ID>*.
  - Scan storage/dumps/ for files matching *<DP-ID>*.
  - Scan storage/_scratch/ for files matching *<DP-ID>*.
- **Exclusion Logic**: The binary must explicitly exclude:
  - .gitkeep files.
  - README.md files.
  - Any file in storage/archives/ (unless a separate rotation policy is invoked).
- **Execution Safety**:
  - If no files match, exit with code 0 and report "No artifacts found for ."
  - If files are found, delete them and report "Removed artifacts for ."
- **Linguistic Constraint**: Output messages must not use contractions (e.g., use "cannot" instead of "can't").

Forensic Insight: This modification transforms prune from a blunt instrument (clearing all trash) to a precise surgical tool, allowing "Clean Room" operations where each DP starts with a pristine workspace, uncontaminated by the artifacts of previous sessions.

5.2 Specification: ops/bin/llms (Integration)

Purpose: To regenerate the "Discovery Map" (llms.txt) and context bundles, ensuring the agent's view of the system is cryptographically current.

Integration Logic:

- **Trigger**: Must be invoked in the Closeout phase if git diff --name-only shows changes to docs/ or PoT.md.
- **Manifest Adherence**: The binary must strictly follow ops/lib/manifests/CONTEXT.md and ops/lib/manifests/LLMS.md.[1]
- **Hazard Prevention**: It must enforce the "Context Hazard" rule by actively rejecting any attempt to bundle docs/library/ files into the global context.[1]
- **Freshness Metadata**: The binary acts as a "Timekeeper" by injecting the current HEAD hash and commit timestamp into the header of llms.txt. This allows future agents to verify if they are looking at a stale map.[1]

5.3 Specification: ops/lib/scripts/harvest (Workflow)

Purpose: To automate the extraction of intellectual capital.

Workflow Logic:

1. **Check Mode**: ops/lib/scripts/agent.sh harvest-check scans SoP.md for repeated tool/pointer clusters.
2. **Draft Generation**: ops/lib/scripts/skill.sh harvest creates a markdown draft in storage/archives/skills/ with a standardized "Provenance" block (Captured date, DP origin, HEAD hash).[1]
3. **Promotion**: ops/lib/scripts/skill.sh promote moves the draft to docs/library/skills/, assigns a canonical ID (e.g., S-LEARN-07), and updates the registry.[1]
4. **Anti-Hallucination**: The script must verify that the new skill does not reference non-existent files ("Ghost Pointers") before allowing promotion.

---

6. Implementation Plan: The Hardened TASK.md Template

The culmination of this research is the "Hardened TASK.md Template." This template replaces the manual checklist with a "Pointer-First" dashboard that drives the agent through the rigorous Closeout Cycle.

Revised TASK.md Structure (Section 4):

4. Closeout (Mandatory)

Pointer: Execute the Closeout Cycle as defined in docs/MANUAL.md.

4.1 Verification Gate

- [ ] **Repo Hygiene:** Run bash tools/verify.sh.[1]
- [ ] **Truth Integrity:** Run bash tools/lint/truth.sh.[1]
- [ ] **Library Guard:** Run bash tools/lint/library.sh.[1]

4.2 Harvest Gate (Memory)

- [ ] **Pattern Check:** Run ops/lib/scripts/agent.sh harvest-check.[1]
- [ ] **Skill Harvest:** (If applicable) Run ops/lib/scripts/skill.sh harvest...
- [ ] **Task Harvest:** (If applicable) Run ops/lib/scripts/task.sh harvest...

4.3 Truth Synchronization

- [ ] **Refresh Bundles:** Run ops/bin/llms to update llms.txt and context bundles.[1]
- [ ] **Verify Refresh:** Check git diff llms.txt to confirm HEAD update.

4.4 Final Log & Prune

- [ ] **Log:** Update SoP.md with DP entry and verification receipts.
- [ ] **Prune:** Run ops/bin/prune --dp=<DP-ID> to clear session artifacts.[1]
- [ ] **Transition:** Append THREAD END entry to TASK.md Work Log.

This template forces the agent to interact with the operational toolchain at every step. It creates a "Chain of Custody" for the work, where the verification receipts, harvest logs, and prune confirmation are all cryptographically linked to the specific Dispatch Packet.

---

7. Strategic Implications and Second-Order Effects

The implementation of this hardening plan generates significant second and third-order effects that extend beyond simple code hygiene.

7.1 The "Immune System" Effect

By integrating tools/lint/library.sh and the harvesting scripts into the closeout cycle, the system develops an "Immune System." Just as biological systems reject foreign bodies, the Stela system will now automatically reject "Ghost Canon" (files in docs/ that are not registered) and "Context Hazards" (library files illegally loaded into the global context). This moves governance from *reactive* (fixing it after it breaks) to *proactive* (preventing the merge).

7.2 The "Pattern Density" Feedback Loop

The use of harvest-check logic creates a positive feedback loop. As the system operates, it generates "Pattern Density"—clusters of repeated actions or tool usages. The harvesting scripts detect these clusters. By mandating harvesting at closeout, we force the system to

acknowledge its own emergent patterns, converting implicit behavior into explicit, reusable Skills. This accelerates the system's learning rate, as future agents can "download" these Skills (via pointers) rather than rediscovering the solution.[1]

7.3 Reducing the "Hallucination Gap"

The rigorous refresh of ops/bin/llms closes the "Hallucination Gap." In many systems, documentation lags behind code. By forcing llms.txt regeneration as a condition of closeout, Stela ensures that the *map* (llms.txt) always matches the *territory* (the repo). This significantly reduces the likelihood of an agent hallucinating a file structure that no longer exists, increasing the reliability of autonomous operations.[1]

7.4 Institutional Memory and "Bus Factor"

In traditional teams, knowledge is lost when key personnel leave (the "Bus Factor"). In Stela, the "Harvesting" protocols ensure that knowledge is captured *as it is created*. The provenance fields in the harvested skills [1] act as a permanent citation, linking the insight back to the specific DP and context where it was formed. This creates a "Self-Documenting History" that is robust against personnel turnover or agent reset.

---

8. Conclusion

The refinement of the Stela system to a "Pointer-First" constitution with an automated, hardened Closeout Cycle represents a paradigmatic shift from "Managed Chaos" to "Industrial Order."

By resolving the SSoT conflict regarding branch naming, we eliminate a critical ambiguity that invites human error and system drift. By automating the hygiene (prune) and memory (harvest) functions, we protect the system from the entropy of accumulated data and uncaptured knowledge.

This plan does not merely "clean up" the repository; it enforces a physical law where the *existence* of a task implies the *maintenance* of the system. The successful implementation of DP-OPS-0046 served as the proof of concept [1]; the integration of prune and llms into the operational heartbeat is the necessary capitalization on that foundation. The system is now poised to operate not just as a tool, but as a self-sustaining intelligence architecture.

---

9. Data Tables

Table 1: System Failure States & Hardening Resolutions (PoT Section 2)

Failure Type	Definition	Impact	Hardening Resolution
Canon Drift	Divergence between PoT.md and repo state.	Agents follow obsolete rules; system instability.	**Pointer-First Constitution**: TASK.md links to PoT.md rather than restating it. **Refresh Cycle**: ops/bin/llms runs at closeout.
Context Hazard	Including library/ docs in CONTEXT.md.	Context window saturation; recursive definition loops.	**Harvest Gate**: ops/lib/scripts/skill.sh check enforces exclusion of library files from manifest.[1]
Dirty State	Uncommitted changes in main.	Loss of proven state; inability to rollback.	**Branching Doctrine**: PoT.md amendment explicitly forbids main pushes.[1]
Ambiguity	Instructions capable of multiple interpretations.	Probabilistic failure in agent execution.	**Linguistic Precision**: ops/bin/prune and other binaries stripped of contractions and ambiguous output.[1]

Table 2: The Refined Closeout Cycle (Sequential Logic)

Stage	Action	Toolchain	Objective
1. Verify	Run Gates	tools/verify.sh tools/lint/truth.sh	Ensure filing doctrine compliance and

		tools/lint/library.sh	library integrity before proceed.
2. Harvest	Capture Knowledge	ops/lib/scripts/agent.sh harvest-check ops/lib/scripts/skill.sh ops/lib/scripts/task.sh	Convert implicit "Pattern Density" into explicit, reusable Skills/Tasks.[1]
3. Refresh	Sync Truth	ops/bin/llms	Regenerate llms.txt and bundles to match current HEAD state.[1]
4. Log	Commit History	SoP.md (Manual Update)	Append DP entry with verification receipts to the immutable ledger.
5. Prune	Clear Waste	ops/bin/prune --dp=<ID>	Remove intermediate OPEN and dump artifacts to restore "Clean Room" state.[1]

Table 3: Branch Naming SSoT Conflict Analysis

Attribute	**PoT.md (Current)**	**.github/copilot-instructions.md**	**PoT.md (Proposed Amendment)**
Authority	"Sole authority across all scopes" [1]	"Guest Contractor Instructions"	"Sole authority..." (Reaffirmed)
Main Branch	Mentions "Dirty State" failure only.	**Hard Rule**: "No Direct Pushes".[1]	**Physical Law**: "Immutable Trunk. Direct pushes forbidden."

Work Branch	Silent.	"All work occurs on work/* branches."	"Work Namespace: work/* is the only valid development scope."
Status	Constitutional Void.	Parallel Jurisdiction (Drift).	**Resolved SSoT**.

---

10. Detailed Forensic Review of Repository Artifacts

10.1 dump-full-main-d14969db Analysis

The dump artifact serves as the primary forensic evidence for this report. It provides a snapshot of the system's "Physical Laws" and "Operational Reality."

- **Filing Doctrine Compliance**: The file list in `` confirms strict adherence to the filing doctrine: ops/ contains binaries, docs/ contains manuals, tools/ contains linters.[1] This validates the "Verify" stage of the proposed closeout cycle.
- **The "Run" Directory**: The ops/ directory contains the executable logic for the system. The presence of ops/bin/context, ops/bin/dump, and ops/bin/llms confirms the existence of the "Ops Kernel" described in docs/ops/README.md.[1]
- **The Missing Prune Spec**: While ops/bin/prune is listed in the index, the absence of its content in the `` block [1] necessitated the inferential analysis in Section 5.1. We derived its required behavior from the "Trash" definition in PoT.md and the "Linguistic Precision" axiom.

10.2 DP-OPS-0046-RESULTS.md Analysis

This result file documents the initial attempt at "Pointer-First" refinement.

- **Achievements**: It successfully "Refined TASK.md to pointer-first contract" and "refreshed llms context bundles".[1]
- **Friction Points**: The report notes that an initial attempt to refresh bundles failed due to a pathing error (--out-dir=.), which was resolved by using an absolute path. This operational learning has been incorporated into the "Refresh" stage of the Hardened Closeout Cycle (Section 4.3).
- **Verification**: The rigorous use of bash tools/lint/* scripts in the verification section of DP-OPS-0046 serves as the template for the "Verify" stage of our new cycle.

10.3 OPEN-main-d14969db.txt Analysis

This artifact represents the "Session State" at the start of a DP.

- **Freshness Gate**: It captures the Active Branch and HEAD hash.[1] This confirms the "Anchor Hygiene" protocol defined in docs/MANUAL.md.
- **Role Contract**: It explicitly defines the AI's stance ("Canon-first, explicit about unknowns").[1] This aligns with the "Behavioral Logic Standard" we are enforcing in the modified prune binary (no contractions).
- **Context Load**: It lists the "Canon Pointers" (PoT.md, TASK.md, etc.). This validates our "Pointer-First" approach; the agent is already trained to look for pointers rather than content in the prompt.

---

11. Appendix: Implementation Code Snippets

11.1 ops/bin/prune Logic (Bash Pseudocode)

Bash

```
#!/usr/bin/env bash
# ops/bin/prune --dp=<DP-ID>
# Enforce Linguistic Precision: No contractions in output.

set -euo pipefail

DP_ID=""
TARGET_DIRS=("storage/handoff" "storage/dumps" "storage/_scratch")

# Argument Parsing
for arg in "$@"; do
  case $arg in
    --dp=*)
      DP_ID="${arg#*=}"
      shift
      ;;
  esac
done

if]; then
  echo "Error: DP ID is required for targeted pruning."
  exit 1
fi
```

```
echo "Pruning artifacts for Dispatch Packet: $DP_ID"

COUNT=0
for dir in "${TARGET_DIRS[@]}"; do
  if [[ -d "$dir" ]]; then
    # Find files matching DP_ID, exclude.gitkeep
    find "$dir" -type f -name "*${DP_ID}*"! -name ".gitkeep" -print0 | while IFS= read -r -d '' file; do
      rm "$file"
      echo "Deleted: $file"
      ((COUNT++))
    done
  fi
done

if]; then
  echo "No artifacts found for $DP_ID."
else
  echo "Removed $COUNT artifacts."
fi
```

11.2 PoT.md Amendment Text (Markdown)

5.5 Branching Doctrine (SSoT)

- **Immutable Trunk**: The main branch is reserved for verified, deployable state. Direct pushes to main are constitutionally forbidden.
- **Work Namespace**: All active development must occur on branches prefixed with work/.
  - **Naming Schema**: Valid formats are work/<dp-id>-<topic> or work/<topic>-YYYY-MM-DD.
- **Drift Prevention**: Any branch not matching this schema is considered "Trash" (per Filing Doctrine) and subject to immediate pruning.

This text must be inserted into PoT.md to resolve the SSoT conflict identified in Section 3.

---

12. Conclusion

The architectural refinements proposed in this report—specifically the **Pointer-First Constitution**, the **Branching Doctrine Amendment**, and the **Hardened Closeout Cycle**—collectively address the root causes of "Drift" and "Ambiguity" in the Stela system. By treating the repository as a living organism that must verify its intake, metabolize its work, and excrete its waste, we ensure its long-term viability and intelligence. The transition from manual checklists to automated, binary-driven gates transforms Stela from a passive storage medium

into an active participant in its own governance.

Works cited

1. dump-full-main-d14969db.txt
2. Structural transparency of societal AI alignment through Institutional Logics. - arXiv, accessed February 10, 2026, https://arxiv.org/html/2602.08246v1
3. AI Governance, accessed February 10, 2026, https://www.governanceinstitute.com.au/app/uploads/2024/09/GovInst-AI-Whitepaper.pdf